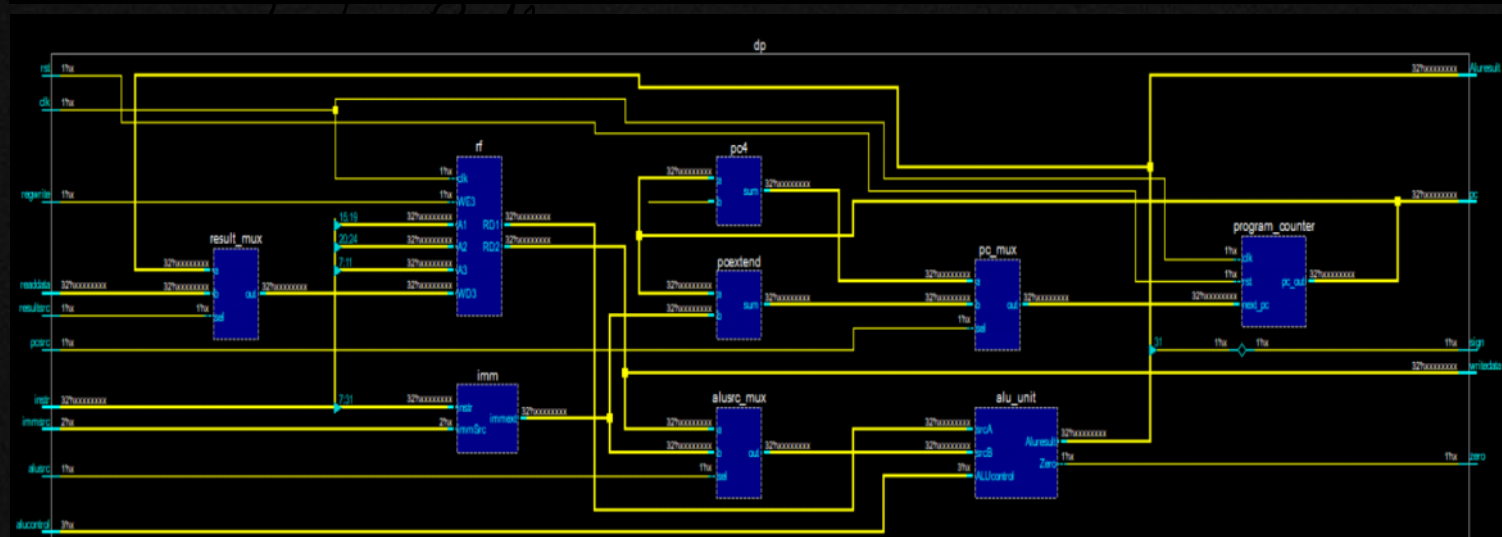
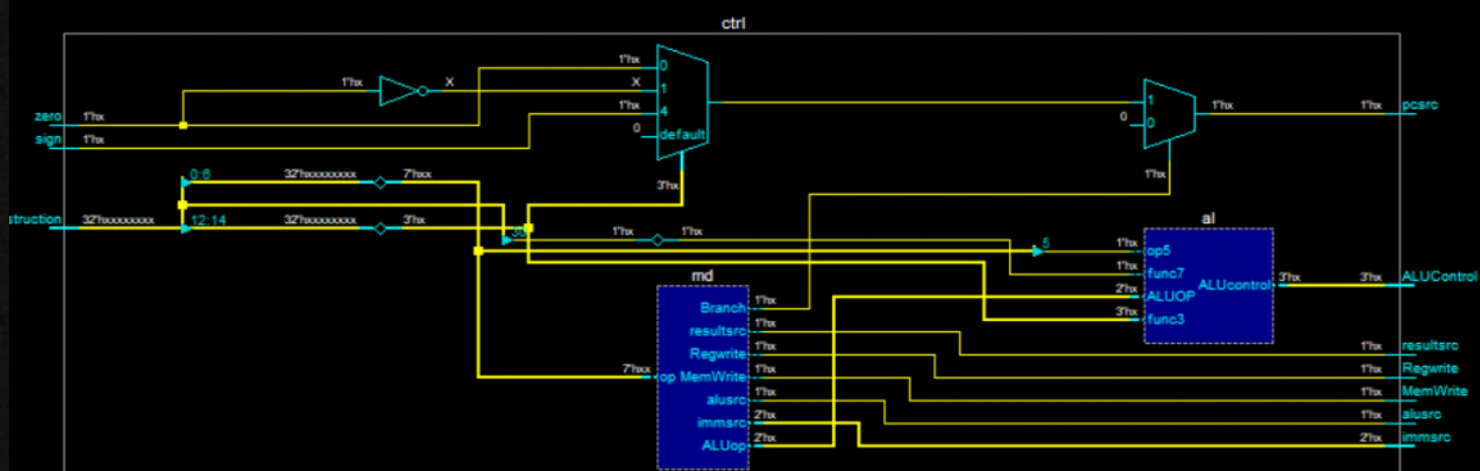
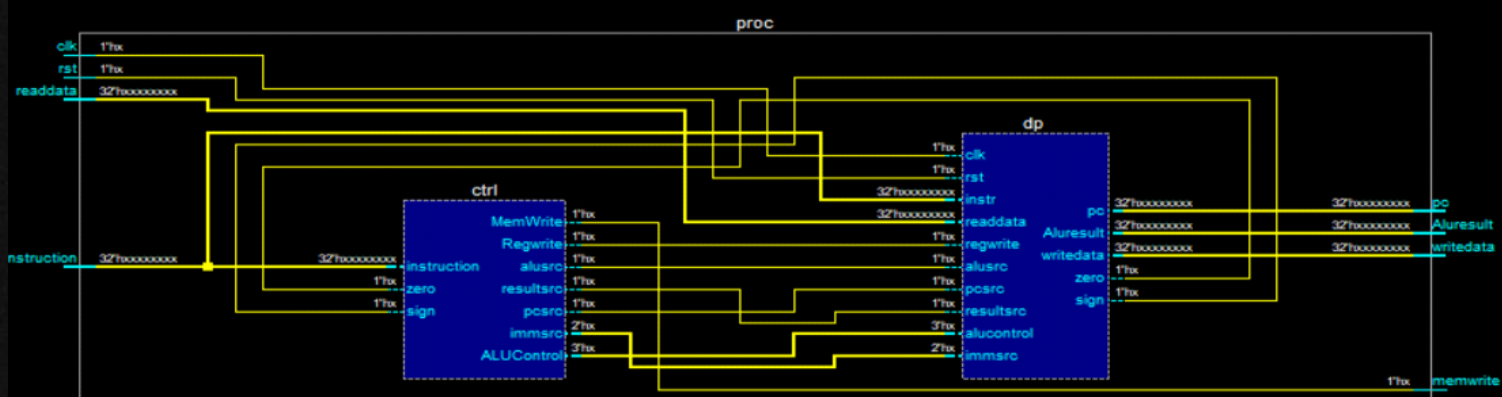
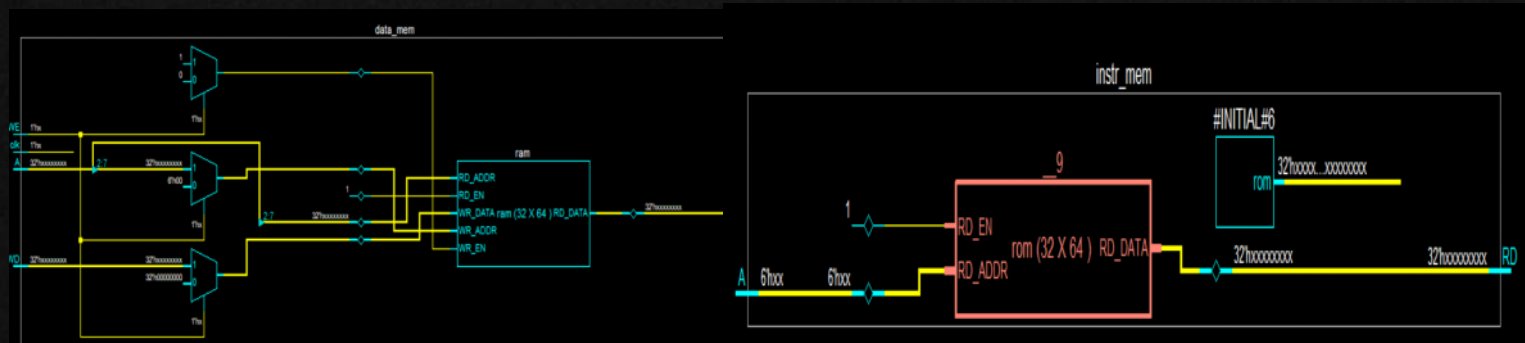




Code

project >  adder.v

```
1  module adder (  
2      input [31:0] a,  
3      input [31:0] b,  
4      output [31:0] sum  
5  );  
6      assign sum = a + b;  
7  endmodule
```

project >  mux.v

```
1  module mux (  
2      input [31:0] a,  
3      input [31:0] b,  
4      input sel,  
5      output [31:0] out  
6  );  
7      assign out = sel ? b : a;  
8  endmodule
```

```
1  module pc ( input clk, rst,  
2      input [31:0] next_pc,  
3      output reg [31:0] pc_out );  
4  always @(posedge clk) begin  
5      if (rst) begin  
6          pc_out <= 0;  
7      end else begin  
8          pc_out <= next_pc;  
9      end  
10 end  
11 endmodule
```



```

module ROM(
    input [5:0] A,
    output [31:0] RD
);
reg [31:0] rom [63:0];
initial begin
    $readmemh("risc.dat", rom);
end
assign RD = rom[A];
endmodule

```

```

module RAM(
    input clk, WE,
    input [31:0] A, WD,
    output [31:0] RD
);
reg [31:0] ram [0:63];
assign RD = ram[A[7:2]];
always @(posedge clk ) begin
    if (WE) begin
        ram[A[7:2]] <= WD;
    end
end
endmodule

```

```

module ALU(
    input [31:0] srcA,srcB,
    input [2:0] ALUcontrol,
    output reg [31:0] Aluresult,
    output Zero
);
always @(*) begin
    case(ALUcontrol)
        3'b000: Aluresult = srcA + srcB; // add
        3'b001: Aluresult = srcA << srcB[4:0]; // sll
        3'b010: Aluresult = srcA -srcB; // sub
        3'b100: Aluresult = srcA ^ srcB; // xor
        3'b101: Aluresult = srcA >> srcB[4:0]; // sr
        3'b110: Aluresult = srcA | srcB; // or
        3'b111: Aluresult = srcA & srcB ; // and
        default: Aluresult = 0;
    endcase
    assign Zero = (Aluresult == 0) ;
end
endmodule

```

```

module extend(
input [31:7]instr,
input [1:0]immSrc,
output reg [31:0] immext
);
always @(*)begin
    case(immSrc)
        2'b00: immext = {{20{instr[31]}},instr[31:20]}; //I type
        2'b01: immext = {{20{instr[31]}},instr[31:25],instr[11:7]}; //S-type
        2'b10: immext = {{20{instr[31]}},instr[7],instr[30:25],instr[11:8],1'b0}; //B-type
        default: immext = 32'b0;
    endcase
end
endmodule

```

```

✓ module regfile (
    input clk,WE3,
    input [4:0]A1, A2,A3,
    input [31:0]WD3,
    output[31:0] RD1,RD2
);
    reg [31:0] regf [0:31];
    assign RD1 = (A1==5'b0)?32'd0:regf[A1];
    assign RD2 = (A2==5'b0)?32'd0:regf[A2];
    always @(posedge clk) begin
        ✓ if (WE3&&A3!=5'b0) begin
            ✓ regf[A3] <= WD3;
        end
    end
end
endmodule

```

```

module aludec(
    input op5,func7,
    input [1:0] ALUOP,
    input [2:0] func3,
    output reg [2:0] ALUcontrol
);
wire funcop;
assign funcop=op5&func7;//op and func7 = 11 for sub, otherwise add
always @(*) begin
    case(ALUOP)
        2'b00: ALUcontrol=3'b000; // add for lw sw
        2'b01: begin
            case(func3)
                3'b000: ALUcontrol=3'b010; // sub for beq
                3'b001: ALUcontrol=3'b010; // sub for bne
                3'b100: ALUcontrol=3'b010; // sub for blt
                default: ALUcontrol=3'b010;
            endcase
        end
        2'b10: begin
            case(func3)
                3'b000: begin
                    if (funcop)
                        ALUcontrol=3'b010; // sub
                    else ALUcontrol=3'b000; // add
                end
                3'b001: ALUcontrol=3'b001; // sll
                3'b100: ALUcontrol=3'b100; // xor
                3'b101: ALUcontrol=3'b101; // srl
                3'b110: ALUcontrol=3'b110; // or
                3'b111: ALUcontrol=3'b111; // and
                default: ALUcontrol=3'b000;
            endcase
        end
        default: ALUcontrol=3'b000;
    endcase
end
endmodule

```



```

module maindec(
    input [6:0] op,
    output Branch , resultsrc ,Regwrite , MemWrite ,alusrc,
    output [1:0] immsrc , ALUop
);
    reg [8:0] control;
    assign {Regwrite,immsrc,alusrc,MemWrite,resultsrc,Branch,ALUop} = control;
    always @(*) begin
        casez(op)
            7'b0000011: control <= 9'b1_00_1_0_1_0_00; //Lw
            7'b0100011: control <= 9'b0_01_1_1_x_0_00; //SW
            7'b0110011: control <= 9'b1_xx_0_0_0_0_10; //R-type
            7'b1100011: control <= 9'b0_10_0_0_X_1_01; //Branch
            7'b0010011: control <= 9'b1_00_1_0_0_0_10; //addi
            default:    control <= 9'bx;
        endcase
    end
endmodule

```

```

module processor (
    input clk, rst,
    input [31:0] instruction, readdata,
    output [31:0] pc, Aluresult, writedata,
    output memwrite
);
    wire regwrite, alusrc, resultsrc, pcsrc,zero, sign ;
    wire [2:0] alucontrol;
    wire [1:0] immsrc;
    controller ctrl (instruction,zero, sign,memwrite, regwrite, alusrc,
    resultsrc,pcsrc,immsrc,alucontrol);
    datapath dp ([clk, rst,instruction, readdata,
    regwrite, alusrc, pcsrc, resultsrc,alucontrol,
    immsrc,pc, Aluresult, writedata,zero, sign
    ]);
endmodule

```

```

module controller (
    input [31:0] instruction, input zero,sign,
    output MemWrite, Regwrite,alusrc, resultsrc,
    output reg pcsrc,output [1:0] immsrc,
    output [2:0] ALUControl
);
    wire [6:0] op; wire [2:0] funct3;wire funct7; wire [1:0] ALUop;
    wire Branch;
    assign op = instruction[6:0];
    assign funct3 = instruction[14:12];
    assign funct7 = instruction[30];
    maindec md (op, Branch , resultsrc ,Regwrite , MemWrite
    ,alusrc,immsrc , ALUop);
aludec al ( op[5], funct7,ALUop,funct3,ALUControl);
    //to calculate pcsrc
    always @(*) begin
        if (Branch) begin
            case (funct3)
                3'b000: pcsrc = zero; // beq
                3'b001: pcsrc = ~zero; // bne
                3'b100: pcsrc = sign; // blt
                default: pcsrc = 1'b0;
            endcase
        end
        else
            pcsrc = 1'b0;
    end
endmodule

```

```

module datapath ([ input clk,rst, input [31:0] instr,readdata,
    input regwrite,alusrc,pcsrc,resultsrc,input [2:0] alucontrol,
    input [1:0] immsrc, output [31:0] pc, Aluresult, writedata,
    output zero,sign]);
    wire [31:0] next_pc, pc_plus4,pc_target, result, imm_ext, srcb, readdata1;
    assign sign = Aluresult[31];
    pc program_counter (clk, rst, next_pc, pc);
    mux pc_mux (pc_plus4, pc_target, pcsrc, next_pc);
    adder pc4 (pc, 32'd4, pc_plus4);
    adder pextend (pc, imm_ext, pc_target);
    extend imm (instr[31:7], immsrc, imm_ext);
    regfile rf (clk,regwrite,instr[19:15], instr[24:20], instr[11:7],
    result, readdata1, writedata);
    mux alusrc_mux (writedata, imm_ext, alusrc, srcb);
    ALU alu_unit (readdata1, srcb, alucontrol, Aluresult, zero);
    mux result_mux (Aluresult, readdata, resultsrc, result);
endmodule

```



```

module top (
    input clk ,rst,
    output [31:0] Aluresult, writedata,
    output memwrite
);
wire [31:0] instruction, readdata, pc;
processor proc (
    clk, rst,
    instruction, readdata,
    pc, Aluresult, writedata,
    memwrite
);
ROM instr_mem (pc[7:2], instruction);
RAM data_mem (
    clk, memwrite,
    Aluresult, writedata,
    readdata
);
endmodule

```

```

`timescale 10ns/1ns
module top_tb;
    reg clk; reg rst;
    wire [31:0] WriteData;
    wire [31:0] ALUResult;
    wire MemWrite;
    // Instantiate DUT
    top dut (clk, rst,  ALUResult,WriteData, MemWrite);
    // Clock generation: 100 MHz (10 ns period)
    always #5 clk = ~clk;

```

```

// Reset + init
initial begin
    clk = 0;
    rst = 1;
    store_count = 0;
    #20 rst = 0;
end

// Monitor memory writes
always @(posedge clk) begin
    if (MemWrite) begin
        $display("STORE %0d: addr=%0d data=%0d time=%0t",
            store_count, ALUResult, WriteData, $time);
    end
end

```

```

// Data check
if (WriteData !== expected[store_count]) begin
    $display("=====");
    $display(" SIMULATION FAILED ");
    $display(" Wrong data at index %0d", store_count);
    $display(" Expected %0d got %0d",
        expected[store_count], WriteData);
    $stop;
end

store_count = store_count + 1;

// PASS condition
if (store_count == 10) begin
    #30;
    $display(" SIMULATION SUCCEEDED ");
    $display(" Fibonacci sequence correct ");
    $display("=====");
    $stop;
end
end

// Timeout safeguard
initial begin
    #5000;
    $display(" SIMULATION TIMEOUT ");
    $display("=====");
    $stop;
end

endmodule

```



```

# STORE 0: addr=0 data=1 time=1450
# STORE 1: addr=4 data=2 time=2150
# STORE 2: addr=8 data=3 time=2950
# STORE 3: addr=12 data=5 time=3650
# STORE 4: addr=16 data=8 time=4450
# STORE 5: addr=20 data=13 time=5150
# STORE 6: addr=24 data=21 time=5950
# STORE 7: addr=28 data=34 time=6650
# STORE 8: addr=32 data=55 time=7450
# STORE 9: addr=36 data=89 time=8150
# =====
# SIMULATION SUCCEEDED
# Fibonacci sequence correct
# =====
# ** Note: $stop      : C:/Users/rmenn/OneDrive/Desktop/project/risc_tb.v(76)
#      Time: 8150 ns  Iteration: 1  Instance: /top_tb
# Break in Module top_tb at C:/Users/rmenn/OneDrive/Desktop/project/risc_tb.v line 76

```

Memory Data - /top_tb/dut/instr_mem/rom	
0000003f	XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX
0000002e	XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX
0000001d	XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX XXXXXXXX fc62cae3 00128293 0013a023 00229393 004181b3 002080b3 00420a63
0000000c	0023a023 00229393 404181b3 00110133 00418c63 00000393 00a00313 00000293 00100213 00100193 00100113 00000093 00004033
fffffffb	

Memory Data - /top_tb/dut/data_mem/ram - Default	
00000000	00000001 00000002 00000003 00000005 00000008 0000000d 00000015 00000022 00000037 00000059 xxxxxxxx xxxxxxxx
00000011	xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx
00000022	xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx
00000033	xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx
00000044	

wave form:

<https://drive.google.com/drive/folders/1M4jBanO4MHPaOq8TsoPHLhjlWb5-FrLt?usp=sharing>